

Software Requirements Specification

Personal Task Manager (CLI Application)

Prepared for: CodeOrbit Tech — Software Development Internship (1 Month Tier)

Prepared by: Subhanshu pal Date: 17-09-2026 Version: 1.0

1. Introduction

1.1 Purpose

This document describes the software requirements for a Personal Task Manager, a command-line application that allows a user to create, view, update, and delete daily tasks. It is prepared as part of the CodeOrbit Tech Software Development Internship (Task 1: Requirement Analysis & SRS Document).

1.2 Scope

The Personal Task Manager is a lightweight, single-user, command-line tool. It allows a user to manage a personal to-do list without requiring a database server, internet connection, or graphical interface. The application stores tasks only for the duration of a session, or optionally to a local file, and does not include multi-user accounts, notifications, or scheduling features in this version.

1.3 Intended Users

- Students and individuals who want a simple way to track personal tasks.
- Beginner developers using the project as a learning exercise in requirements analysis and CLI development.

1.4 Definitions and Abbreviations

- SRS – Software Requirements Specification
- CLI – Command-Line Interface
- FR – Functional Requirement
- NFR – Non-Functional Requirement

2. Overall Description

2.1 Product Perspective

The Personal Task Manager is a standalone application. It does not depend on any external system or service. It reads user commands from the terminal and displays output directly in the terminal.

2.2 Assumptions and Constraints

- The application does not require user authentication (login/signup) in this version.
- Data may be stored only in memory during a session, or optionally in a simple local file (e.g., JSON or text file) for persistence between runs.
- The application is single-user; no concurrent access is expected.
- The user has Python (or the chosen language's runtime) installed on their machine.

3. Functional Requirements

The table below lists the core functional requirements the system must satisfy.

ID	Requirement
FR-1	The system shall allow the user to add a new task with a text description.
FR-2	The system shall display all tasks in a numbered list with their current status.
FR-3	The system shall allow the user to mark a task as completed by selecting its ID.
FR-4	The system shall allow the user to delete a task by selecting its ID.
FR-5	The system shall validate all user input and reject invalid menu choices without crashing.
FR-6	The system shall allow the user to exit the application safely at any time.

4. Non-Functional Requirements

The table below lists quality attributes the system must meet.

ID	Category	Requirement
NFR-1	Usability	The menu shall be simple, numbered, and require no prior training to use.
NFR-2	Performance	All operations (add, view, delete, mark complete) shall respond within 1 second.
NFR-3	Reliability	The application shall not crash on invalid input; it shall show an error and re-prompt.
NFR-4	Portability	The application shall run on any system with Python 3 installed, with no external dependencies.
NFR-5	Maintainability	Code shall be modular (separate functions per feature) and commented for readability.

5. Use Cases and User Stories

5.1 Use Case Summary

ID	Use Case	Description
UC-1	Add Task	User enters a task description; system stores it with status 'Pending'.
UC-2	View Tasks	User views a list of all tasks with their status (Pending/Completed).
UC-3	Mark Task Complete	User selects a task by ID; system updates its status to 'Completed'.
UC-4	Delete Task	User selects a task by ID; system removes it from the list.
UC-5	Search Task	User enters a keyword; system displays matching tasks.
UC-6	Exit Application	User selects exit; system closes safely, saving any pending data.

5.2 Sample User Stories

- As a user, I want to add a new task so that I can keep track of things I need to do.
- As a user, I want to view all my tasks so that I know what is pending and what is completed.
- As a user, I want to mark a task as complete so that I can track my progress.
- As a user, I want to delete a task so that I can remove items I no longer need.
- As a user, I want the application to reject invalid input gracefully so that it never crashes on a mistake.

6. Basic System Flow

The application presents a numbered menu on each run of the main loop:

- 1. Add Task
- 2. View Tasks
- 3. Mark Task Complete
- 4. Delete Task
- 5. Search Task
- 6. Exit

The user selects an option by entering its number. The system validates the input, performs the corresponding action, and returns to the menu until the user chooses to exit.

7. Future Scope (Out of Current Scope)

- Persistent storage using a database (e.g., SQLite).
- Multi-user support with login/authentication.
- Due dates, priorities, and reminders/notifications.
- A graphical or web-based user interface.

8. Document Sign-off

This SRS represents the agreed scope for the Task 1 deliverable of the CodeOrbit Tech Software Development Internship (1 Month Tier).